

06/02/26

exploration algo dfs - ikram benchalal (screenshots des résultats à la fin de ce pdf)

=====

note : cet algo est en cours d'exploration, mes collegues (aya et nada) et moi testons encore d'autres approches avant de faire un choix definitif

ce que j'ai fait

j'ai implementé un generateur de labyrinthe basé sur l'algorithme DFS (depth-first search / recursive backtracker) avec ajout de boucles pour obtenir un labyrinthe jouable style pac-man.

representation du labyrinthe

- le labyrinthe est une matrice 2D (liste de listes en python)
- chaque case vaut 0 (mur) ou 1 (couloir)
- les dimensions doivent etre impaires pour que l'algo fonctionne bien

fonctionnement de l'algo dfs

1. on demarre sur une case (1,1) qu'on marque comme couloir
2. on choisit une direction aleatoire parmi haut/bas/gauche/droite
3. si la case 2 pas plus loin est un mur non visité :
 - on casse le mur entre les deux cases
 - on avance et on recommence (recursion)
4. si toutes les directions sont bloquées, on revient en arriere (backtrack)
5. quand tout est exploré, le labyrinthe de base est terminé

resultat de dfs seul : labyrinthe "parfait" (un seul chemin entre 2 points)

ajout de boucles

pour pac-man on veut plusieurs chemins possibles, donc apres la generation :

- je parcours tous les murs interieurs
- je garde ceux qui separent deux couloirs
- j'en supprime un pourcentage (parametre loop_percent)

ca cree des boucles = le joueur peut echapper aux fantomes par plusieurs routes

complexité

- dfs : $O(n)$ ou n = nombre de cases (on visite chaque case une fois)
- ajout de boucles : $O(n)$ pour parcourir les murs + $O(k)$ pour en supprimer k
- complexité totale : $O(n)$

- memoire : $O(n)$ pour stocker la matrice + pile de recursion

parametres modifiables

- width, height : taille du labyrinthe
- loop_percent : pourcentage de murs a supprimer (10% = difficile, 40% = facile)

avantages de cet algo

- simple a comprendre et implementer
- genere des labyrinthes avec des couloirs longs et sinueux
- facile a rendre imparfait (ajout de boucles)
- complexité lineaire

inconvenients potentiels

- peut creer des couloirs tres longs sans intersection
- la recursion peut poser probleme pour de tres grands labyrinthes (stack overflow)

note : on explore encore d'autres algos (kruskal, prim, recursive division)
pour comparer les resultats. le choix final sera fait avec aya et nada apres
nos tests respectifs

